



A ferroelectric–memristor memory for both training and inference

In the format provided by the
authors and unedited

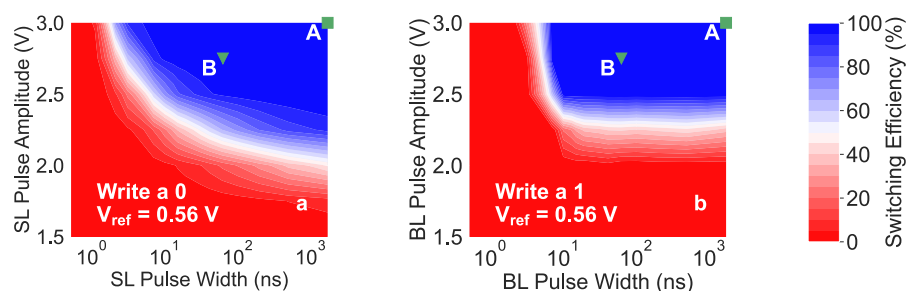
Supplementary information: A ferroelectric–memristor memory for both training and inference

Supplementary Note 1: FeCAPs switching efficiency at array level

In this Note, we detail the measurement of switching efficiency for ferroelectric capacitors (FeCAPs) at the array level. After performing wake-up cycling with 10^3 pulses, we measured the switching efficiency on the 16 kilo-device array across various pulse amplitudes and durations. The fixed reading conditions were set to $V_{ref} = 0.56$ V and 3 V-2 μ s pulses to identify the optimal programming conditions for the FeCAPs.

Suppl. Fig. 1 illustrates the trade-off between pulse width and amplitude for programming a zero (source line pulse) or a one (bit line pulse) in cells initially programmed to the opposite state. The two programming conditions highlighted in the Results section – A and B – both achieved 100% switching efficiency.

All these measurements relied on on-chip embedded delay blocks, designed to generate pulse timings down to 520 ps.



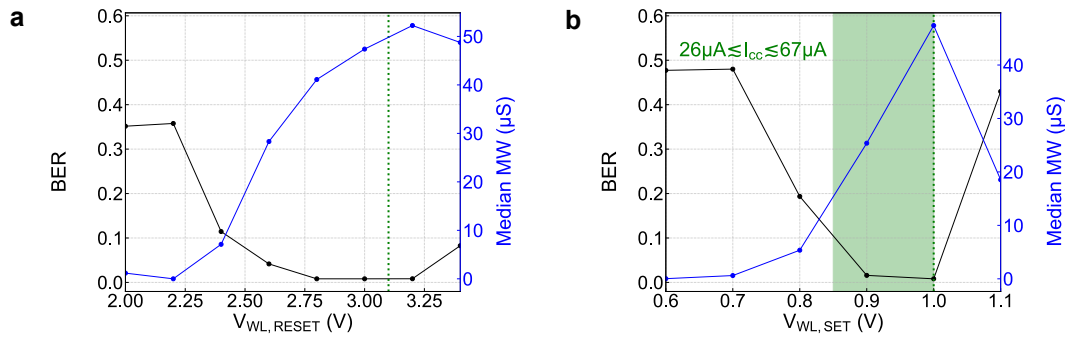
Supplementary Figure 1. Impact of the FeCAP programming conditions on the switching efficiency. The percentage of correctly programmed devices is evaluated for several pulse amplitudes and durations. This was measured after wake-up cycling with 10^3 pulses (3 V-2 μ s). Programming conditions A and B are highlighted by a square and triangular symbol respectively, both showing 100% reliability.

Supplementary Note 2: Programming conditions of memristors

In this Note, we detail the programming conditions used to optimize the performance of memristors in our 16 kilo-device array. Following the forming step, we conducted cycling tests under various programming conditions to minimize the bit error rate (BER) and enhance the memory window.

Suppl. Figs. 2a and 2b illustrate the BER and the median memory window as functions of the word line voltage applied during the reset and set operations, respectively. The optimal reset condition is achieved at $V_{BL} = 3.0$ V and $V_{WL} = 3.1$ V, while the optimal set condition for binary operation is obtained at $V_{SL} = 2.3$ V and $V_{WL} = 1.0$ V. For analog programming, V_{WL} can be modulated within the range of 0.85 V to 1.0 V. We employed 1 μ s long pulses for both set and reset operations.

Suppl. Table 1 summarizes the programming conditions used to program the memristor devices in a multi-level fashion, as shown in Fig. 2g in the Results section.



Supplementary Figure 2. Impact of the memristor programming conditions on the bit error rate and the memory window. Bit error rate and median memory window as function of the gate voltage applied to the selector transistor during set (a) and reset (b) operations. The green area represents the conditions for analog programming.

Level	I_{prog} (μA)	G_{min} (μS)	G_{max} (μS)
1	24.4	18	20
2	30.8	24	26
3	37.3	30	32
4	43.8	36	38
5	50.3	42	44
6	56.8	48	50
7	63.2	54	56
8	69.7	60	62

Supplementary Table 1. Programming conditions for analog programming.

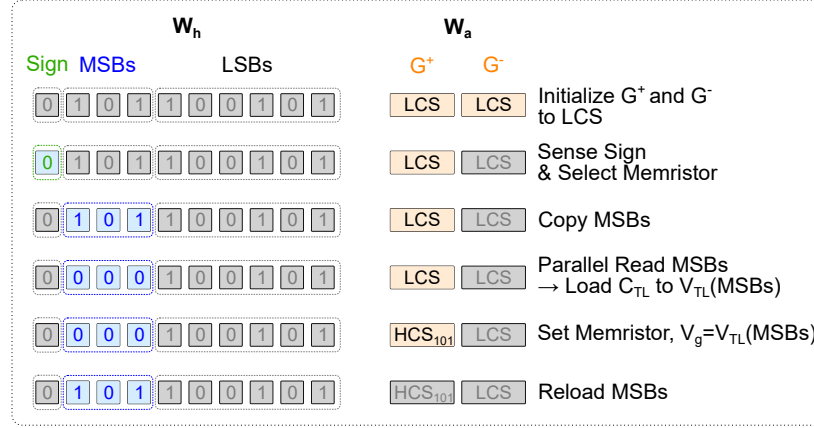
Supplementary Note 3: Detailed data transfer with FeCAPs/memristors memory circuit

In this Note, we provide a detailed example of a data transfer operation from the FeCAPs to the analog memristors in our memory circuit, illustrated in Suppl. Fig. 3. The hidden weight, stored in a sign-and-magnitude integer format utilizing ten FeCAPs, should be transferred to two memristor devices in a differential configuration, enabling the storage of both positive and negative weights.

The process involves the following steps:

1. **Reset the Memristor Cells:** The first step is to reset the two memristor cells to the low conductance state (LCS).
2. **Read the Sign Bit:** The sign bit stored in the first FeCAP cell is read to determine which memristor cell should be programmed. In this example, the positive memristor cell is selected.
3. **Store the MSBs Temporarily:** The three most significant bits (MSBs) from the FeCAP cells are temporarily stored elsewhere. Only three additional memory bits are required for this operation.
4. **Read the MSB FeCAP Cells in Parallel:** The MSB FeCAP cells are read in parallel, which loads the transfer line parasitic capacitance to a voltage level determined by the data stored in the three FeCAPs.
5. **Program the Selected Memristor Cell:** The selected memristor cell is programmed to the high conductance state, with a compliance current determined by the transfer line voltage. During this operation, the data stored in the FeCAPs are lost, resulting in all devices being set to zero.

6. **Reload the MSBs:** The MSBs, which were previously stored elsewhere, should be reloaded after the transfer operation. This reloading is necessary to resume training after the transfer operation in the same configuration as before updating the memristor's differential pair.



Supplementary Figure 3. Example of data transfer operation. Example of a data transfer operation from the FeCAPs to the analog memristors, elucidating all the steps the FeCAP and memristors devices undergo.

Supplementary Note 4: Optimizing FeCAP area ratios for precise transfer line voltage level control

In this Note, we explore the optimization of FeCAP area ratios to ensure precise control of transfer line voltage levels. Different FeCAPs are used with varying areas to account for the significance of each MSB. Specifically, the ratio between the areas of two adjacent FeCAPs is set to two. This configuration allows the combined state of the n MSBs stored in these capacitors to generate 2^n distinct voltage levels on the transfer line, V_{TL} . Each combination of MSBs corresponds to a unique voltage level with near-linear spacing between the V_{TL} values, which is essential for programming the memristors. This result can be demonstrated analytically.

For n capacitors connected to the same transfer line, the V_{TL} can be expressed as

$$V_{TL} = \frac{\sum_{i=0}^{n-1} C_i}{\sum_{i=0}^{n-1} C_i + C_{TL}} V_{SL}, \quad (1)$$

where C_i is the capacitance of the i -th ferroelectric capacitor, C_{TL} the transfer line parasitic capacitance and V_{SL} the applied read voltage on the source line.

The capacitance of the FeCAPs is the sum of a dielectric and ferroelectric component

$$C_i = C_{D,i} + C_{F,i} = \epsilon_0 \epsilon_r \frac{S_i}{d} + \delta_i \frac{2P_R S_i}{V_{SL}}, \quad (2)$$

where S_i and d are the area and thickness of the i -th FeCAP, ϵ_0 is the vacuum permittivity, ϵ_r the relative permittivity of the ferroelectric material, P_R is the remanent polarization of the capacitor, and δ_i is equal to one or zero depending on the state stored into the ferroelectric capacitor. Under the approximation:

$$\sum_{i=0}^{n-1} C_{F,i} \ll \sum_{i=0}^{n-1} C_{D,i} + C_{TL}, \quad (3)$$

equation (1) can be rewritten as:

$$V_{TL} \approx \frac{\sum_{i=0}^{n-1} C_{D,i} + \sum_{i=0}^{n-1} C_{F,i}}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} V_{SL}. \quad (4)$$

By combining Equation (2) and (4)

$$V_{TL} = \frac{\sum_{i=0}^{n-1} C_{D,i} + \sum_{i=0}^{n-1} C_{F,i}}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} V_{SL} = V_{TL,D} + \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} \sum_{i=0}^{n-1} \delta_i S_i. \quad (5)$$

The transfer line voltage is the sum of a fixed dielectric component $V_{TL,D}$ and a component dependent on the data stored into the ferroelectric capacitors.

As an example, we analyze the case $n=2$, the V_{TL} can assume 4 different values depending on the stored data:

$$V_{TL,00} = V_{TL,D} \quad (6)$$

$$V_{TL,01} = V_{TL,D} + \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_0 \quad (7)$$

$$V_{TL,10} = V_{TL,D} + \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_1 \quad (8)$$

$$V_{TL,11} = V_{TL,D} + \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} (S_0 + S_1). \quad (9)$$

To ensure equal spacing between V_{TL} values corresponding to adjacent bit combinations in the FeCAPs, we imposed that the $V_{TL,01} - V_{TL,00} = V_{TL,10} - V_{TL,01}$. In this way we obtain:

$$\frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_0 = \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_1 - \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_0 \quad (10)$$

which simplifies to

$$S_1 = 2S_0. \quad (11)$$

By setting $S_0 = S_{min}$, the four V_{TL} levels are

$$V_{TL,00} = V_{TL,D} \quad (12)$$

$$V_{TL,01} = V_{TL,D} + \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_{min} \quad (13)$$

$$V_{TL,10} = V_{TL,D} + 2 \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_{min} \quad (14)$$

$$V_{TL,11} = V_{TL,D} + 3 \frac{2P_r}{\sum_{i=0}^{n-1} C_{D,i} + C_{TL}} S_{min}. \quad (15)$$

In conclusion, if the two FeCAPs have areas $S_1 = 2S_0$ four equally spaced transfer line voltage levels are obtained. This approach easily extends to larger values of n .

Supplementary Note 5: FeCAPs bit-cell design utilizing 3-D capacitors to enhance scalability

In this note, we investigate the scalability of the proposed hybrid memory circuit using high aspect ratio 3-D ferroelectric capacitors and compare the proposed solution with SRAM. A critical element concerning scalability is the hardware implementation of bitcells with four times the minimum area and two times the minimum area to store the MSBs to ensure near-linear spacing between the transfer line voltage values and the different bit combinations, as demonstrated in Supplementary Note 4. We propose a unified FeCAP bitcell that integrates up to four 3-D ferroelectric cylindrical capacitor structures on the same selector transistor to achieve different equivalent areas without increasing the bitcell's actual area.

Suppl. Fig. 4a illustrates the main process steps for fabricating 3-D capacitors. The critical steps include the cylinder etching and the conformal deposition of both the electrodes and the ferroelectric layer.

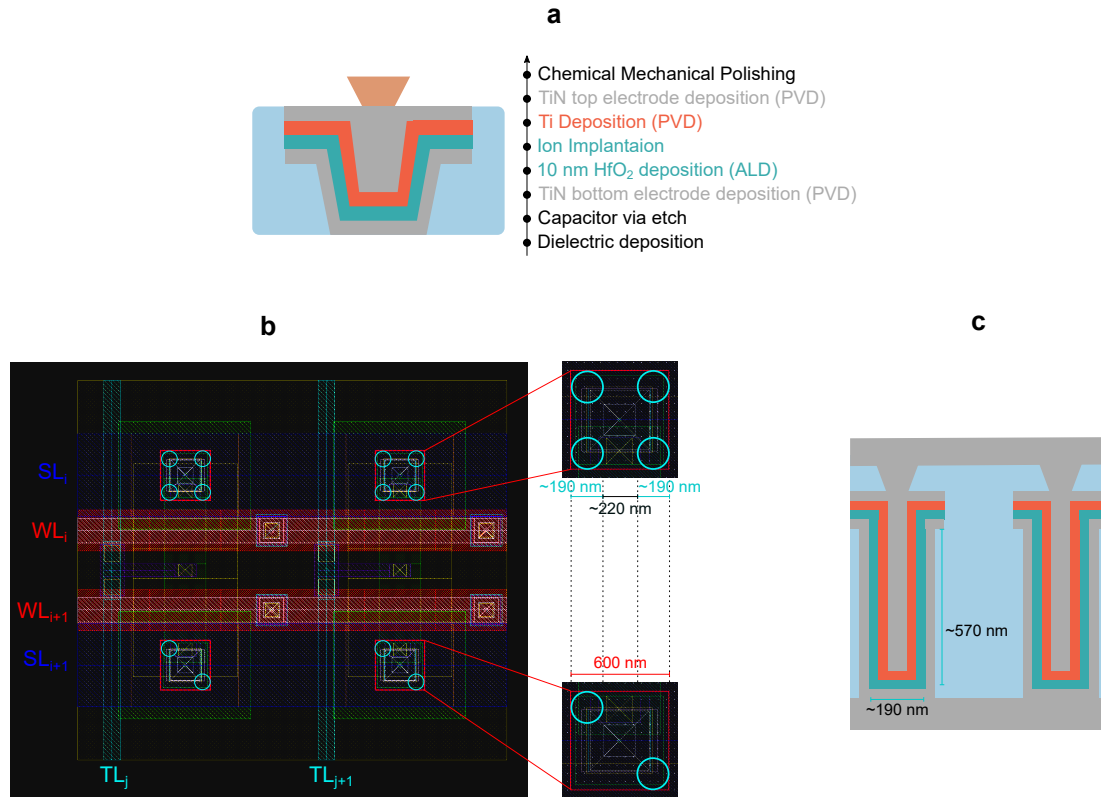
3-D ferroelectric capacitors have been demonstrated at the 130 nm technology node, with an aspect ratio of three, indicating the ratio between the cylinder height and its diameter¹. In the following analysis, we imposed an aspect ratio of three. It is important to note that this imposed aspect ratio is quite conservative: Larger aspect ratios, ranging between 15 and 20, have been achieved at more advanced technology nodes, suggesting the possibility of further increasing the integration density².

For an aspect ratio of three, to achieve an area equivalent to the one in the measured array in Fig. 3e in the Results section, a cylindrical capacitor with a radius of 190 nm and a height of 570 nm should be fabricated, as shown in Suppl. Fig. 4b. Neglecting the flange contribution to the area of the cylindrical capacitor, capacitors with area equal to four and two times that of the minimum-sized one can be obtained by connecting multiple minimum-area capacitors in parallel in the BEOL. As shown in Suppl. Fig. 4b, four capacitors fit in the same space occupied by a planar minimum area capacitor structure, with a spacing between the 3D structures of 220 nm. Suppl. Fig. 4c shows the side view of the capacitors to be fabricated over the same access transistor, sharing the same top and bottom electrode for parallel connection of the minimum surface 3-D structures.

This solution allows to keep the same total bitcell footprint, while we propose to use 1T4C, 1T2C, and 1T1C cells to store bits with different significance. In other words, each “1TnC” cell remains bounded by the same planar footprint, with n cylindrical capacitors stacked vertically. This eliminates any large area penalty from having bigger plate areas for MSBs.

An alternative approach to get “equivalent larger” capacitors for MSBs without using asymmetric size cells is to combine multiple 1T–1C cells in parallel. This approach results in an area penalty: For example, storing ten bits (one sign bit + nine bits of magnitude) requires fourteen FeCAP cells in total, thereby lowering overall bit density. For reference, if a particular 3D FeCAP technology achieves 0.45Gb/mm² (as in Ref.²), then substituting multiple 1T–1C cells in place of 3D designs would drop the density to about 0.32Gb/mm². Nonetheless, even 0.32Gb/mm² remains far above the bit density of typical SRAM in advanced nodes. For comparison, TSMC's 3nm FinFET SRAM bitcell reaches $\approx 33.5\text{Mb/mm}^2$, which is more than an order of magnitude lower than even the “non-optimized” FeCAP approach.

Regarding speed, SRAM has the lowest read/write latency among all memory technologies, approximately ten times faster than the most advanced ferroelectric memories (see Fig. 3 in Ref.²). However, due to its lower density, SRAM-based systems may require frequent data reloading from a main memory, increasing overall latency and power consumption. Additionally, as a volatile memory, SRAM incurs standby power consumption, whereas non-volatile ferroelectric memories consume no power in standby mode.



Supplementary Figure 4. Unified memory bitcell with 3-D capacitors. **a** Process flow highlighting the key steps in the fabrication of 3-D ferroelectric cylindrical capacitors. **b** Bitcell layout with equal $0.36 \mu\text{m}^2$ planar capacitors (red squares) and potential parallel-connected 3-D capacitors (teal circles) with cylinder diameter of 190 nm and aspect ratio of three, for an open cylinder area of $0.37 \mu\text{m}^2$. Either four or two 3-D capacitors can be connected in parallel to achieve larger effective surfaces to store the MSBs of the proposed hybrid memory circuit. **c** Schematic side view of the cylindrical capacitors fabricated over the same selector transistor. The top and bottom electrodes are shared to connect either four or two capacitors in parallel.

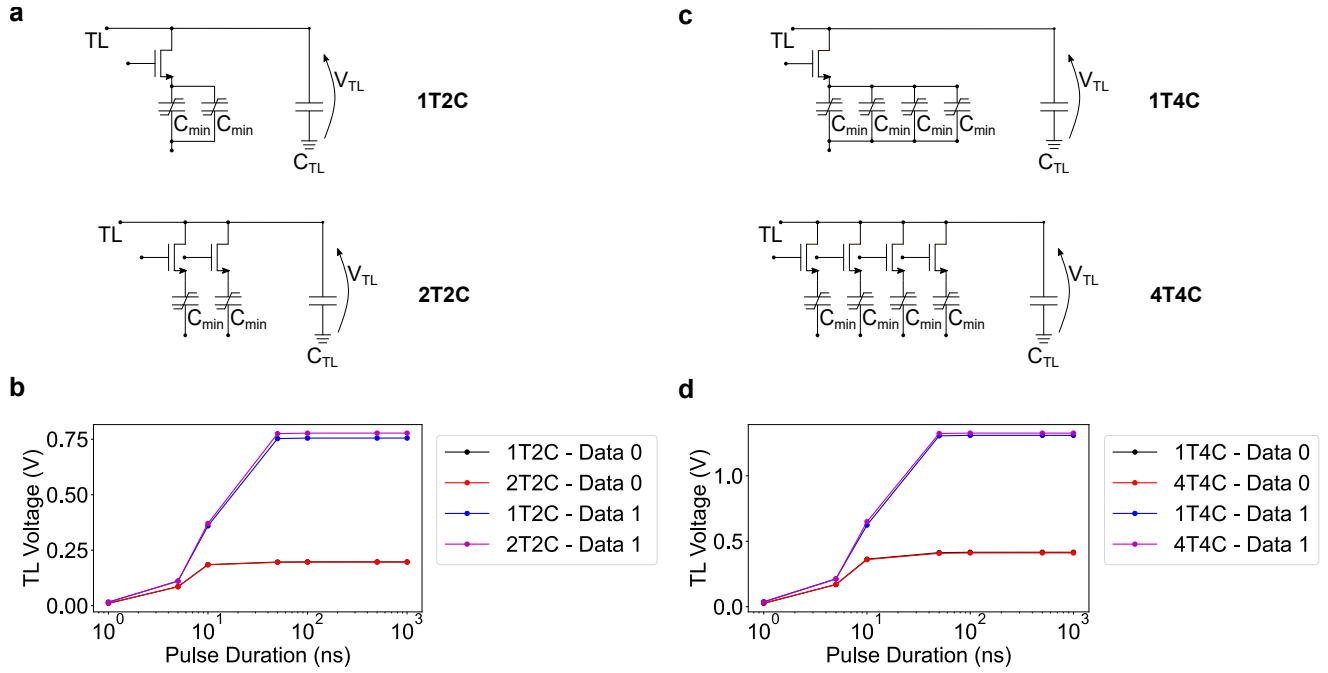
Supplementary Note 6: Analyzing the impact of selector transistors on nT-nC and 1T-nC cells

In this Note, we investigate the impact of selector transistors on 1T-1C cells with varying FeCAP areas used to store hidden weights in a sign and magnitude 10-bit integer format. The three MSBs are stored in capacitors of sizes $4S_{\min}$, $2S_{\min}$, and $S_{\min}$ to ensure near-linear spacing between the transfer line voltage values and the different bit combinations, as demonstrated in Supplementary Note 4. Since each FeCAP bit cell in the fabricated array (Fig. 3a in the Results section) is a 1T-1C cell, we stored the MSBs in the bit cells using 1T-1C, 2T-2C, and 4T-4C configurations.

We performed SPICE simulations to check if the supplementary selector transistors in the manufactured array affect the transfer line voltages achieved during the transfer operation. Specifically, we simulated and compared the circuits in Suppl. Fig. 5a and Suppl. Fig. 5c for several durations of parallel read pulses applied to the 1T-2C/2T-2C and 1T-4C/4T-4C cells. Prior to the parallel read operation, FeCAP devices were programmed to either one or zero by applying 1 μ s long pulses. The pulse amplitude for read and write operations was set to 4.8 V. Thick gate oxide transistors with a length and width of 500 nm were used as selector transistors. A fixed transfer line capacitance $C_{TL}=200$ fF was utilized.

For simulations, we employed a Verilog-A model for the FeCAPs, modeling devices with a remanent polarization of 19 μ C/cm², a saturation polarization of 20 μ C/cm², a coercive field of 1.5 MV/cm, a leakage resistance of 1 M Ω , a capacitor thickness of 10 nm, a capacitor surface area of 0.25 μ m², and a relative permittivity of the ferroelectric material of 30.

The simulation results shown in Suppl. Fig. 5b for the comparison of the 1T-2C and 2T-2C cells, as well as the results for the 1T-4C and 4T-4C cells comparison in Suppl. Fig. 5d, indicate a negligible difference in the transfer line voltage levels for sufficiently long read pulse durations. For read pulses of 1 μ s, the worst-case difference between the transfer line voltage levels achieved with single or multiple access transistors is approximately 20 mV, confirming that the 1T-2C cells and 1T-4C cells can be reliably emulated by our fabricated array with minimal-area 1T-1C cells.



Supplementary Figure 5. Comparison of 1T-2C vs 2T-2C cells and 1T-4C vs 4T-4C cells. **a** Schematic of 1T-2C and 2T-2C cells connected to transfer lines with parasitic capacitance C_{TL} . **b** Electrical simulations of the transfer line voltage as function of the parallel read pulse duration for capacitors programmed to either zero (Data 0) or one (Data 1) for the 1T-2C/2T-2C cells. **c** Schematic of 1T-4C and 4T-4C cells connected to transfer lines with parasitic capacitance C_{TL} . **d** Electrical simulations of the transfer line voltage as a function of the parallel read pulse duration for capacitors programmed to either zero (Data 0) or one (Data 1) for the 1T-4C/4T-4C cells.

Supplementary Note 7: Pseudo-code for training fully-connected neural networks for multi-class classification with FeCAPs/memristors circuit

This note details the different training algorithms used throughout this work.

Algorithm 1: Train an ANN (online hybrid-memory implementation) with L layers. The function *LayerNorm()* specifies how to layer-normalize the activations. *LayerNormBackward()* specifies how to backpropagate through the normalization. *ReLU()* and *Softmax()* specify how to apply respectively ReLU and Softmax functions to the activations. *ReLUBackward()* specifies how to backpropagate through ReLU. *Update()* specifies how to update the weights when their gradients are known, function described in Algorithm 2. *Quantize()* and *Transfer()* functions (Algorithms 3 and 4 respectively) specify how to quantize the hidden weights or evaluate analog weights respectively. k is the counter limit after which the analog/quantized weights are updated, η is the learning rate, p is the hidden weights update probability, *VariabilityFlag* defines whether variability during transfer is considered.

Require: One input and target (a^0, a^*), previous hidden weights ($W^{(h)}$), previous analog weights ($W^{(a)}$), counter state (*count*)

Ensure: Updated weights ($W^{(h),t+1}$ and $W^{(a),t+1}$), updated counter ($count^{t+1}$)

{ 1. Computing the parameters gradients }

{ 1.1 Forward Propagation: }

```
for  $i = 1$  to  $L$  do
     $s_i \leftarrow W_i^{(a)} a_{i-1}$ 
     $a_i, \mu_i, \sigma_i \leftarrow LayerNorm(s_i)$ 
    if  $i < L$  then
         $a_i \leftarrow ReLU(a_i)$ 
    else
         $a_i \leftarrow SoftMax(a_i)$ 
    end if
end for
```

{ 1.2 Backward Propagation: } for $i = L$ to 1 do

```
    if  $i = L$  then
         $g_{a_i} \leftarrow a_i - a^*$ 
    else
         $g_{a_i} \leftarrow ReLUBackward(g_{a_i}, a_i)$ 
    end if
     $g_{s_i} \leftarrow LayerNormBackward(g_{a_i}, s_i, \mu_i, \sigma_i)$ 
     $g_{a_{i-1}} \leftarrow W_i^{(a)\top} g_{s_i}$ 
     $g_{W_i} \leftarrow g_{s_i} a_{i-1}^\top$ 
end for
```

{ 2. Accumulating the parameters gradients }

```
for  $i = 1$  to  $L$  do
     $W_i^{(h),t+1} \leftarrow Update(W_i^{(h)}, \eta, p, g_{W_i})$ 
    if  $count = k$  then
        if VariabilityFlag = True then
```

```


$$W_i^{(a),t+1} \leftarrow Transfer(W_i^{(h),t+1})$$

else

$$W_i^{(a),t+1} \leftarrow Quantize(W_i^{(h),t+1})$$

end if
end if

$$count^{t+1} = count + 1$$

end for

```

Algorithm 2: Update integer hidden weights ($W^{(h)}$) of one ANN layer (online hybrid-memory implementation).

Binomial() specifies how to generate a binary mask for a given layer. *Clip()* specifies how to clip the weights. *Round()* specifies how to round the integer weights to the closest integer value. η is the learning rate, p is the hidden weights update probability, N_h is the number of bits of the hidden weights, j is the layer index. $\circ$ denotes the element-wise multiplication.

Require: Previous hidden weights ($W^{(h)}$) and weights gradients (g_W)

Ensure: Updated hidden weights ($W^{(h),t+1}$)

```


$$M \leftarrow Binomial(p, j)$$


$$W^{(h),t+1} \leftarrow Clip(Round((W^{(h)} - \eta g_W \circ M)2^{N_h-1}), 1 - 2^{N_h-1}, 2^{N_h-1} - 1)/2^{N_h-1}$$


```

Algorithm 3: Quantize integer hidden weights ($W^{(h)}$) of one ANN layer to their low bit-version ($W^{(a)}$). *Sign()* specifies the sign of the weight. *Trunc()* returns the greatest integer less than or equal to its input. *Abs()* evaluates the absolute value. N_h is the number of bits of the hidden weights and N_a is the number of bits of the analog weights.

Require: Previous hidden weights ($W^{(h)}$)

Ensure: Updated analog weights ($W^{(a),t+1}$)

```


$$W^{(a),t+1} \leftarrow Sign(W^{(h)})Trunc(Abs(W^{(h)})2^{N_a-1})/2^{N_a-1}$$


```

Algorithm 4: Transfer integer hidden weights ($W^{(h)}$) of one an ANN to their analog version ($W^{(h)}$) considering transfer operation variability. *Quantize()* performs the operations described in *Algorithm 3*. *Round()* specifies how to round the integer weights to the closest integer value. *Normal()* samples random numbers from a probability distribution for a given layer. *Sign()* specifies the sign of the weight. *Clip()* specifies how to clip the weights. N_h is the number of bits of the hidden weights and N_a is the equivalent number of bits of the analog weights. j is the layer index, μ and σ are the average and standard deviation of the different analog weights levels.

Require: Previous hidden weights ($W^{(h)}$)

Ensure: Updated analog weights ($W^{(a),t+1}$)

```


$$x \leftarrow Quantize(W^{(h)})2^{N_a-1}$$


$$y \leftarrow \sigma Normal(j) + \mu$$


$$Mask \leftarrow Sign(x)Sign(y) > 0$$


$$W^{(a),t+1} \leftarrow y \cdot Mask / 2^{N_a-1}$$


```

Algorithm 5: Train an ANN (minibatch hybrid-memory implementation) with L layers. The function *BatchNorm()* specifies how to batch-normalize the activations. *BatchNormBackward()* specifies how to backpropagate through the normalization. *ReLU()* and *Softmax()* specify how to apply respectively ReLU and Softmax functions to the activations. *ReLUBackward()* specifies how to backpropagate through ReLU. *Update()* specifies how to update the weights when their gradients are known,

function described in Algorithm 2. *Quantize()* and *Transfer()* functions (Algorithms 3 and 4 respectively) specify how to quantize the hidden weights or evaluate analog weights respectively. k is the counter limit after which the analog/quantized weights are updated, η is the learning rate, p is the hidden weights update probability, *VariabilityFlag* defines whether variability during transfer is considered, ε is the moving average coefficient.

Require: One input and target (a^0, a^*), previous hidden weights ($W^{(h)}$), previous analog weights ($W^{(a)}$), counter state (*count*), previous moving average values (μ_{test} and σ_{test}).

Ensure: Updated weights ($W^{(h),t+1}$ and $W^{(a),t+1}$), updated counter ($count^{t+1}$), updated moving averages for test (μ_{test}^{t+1} and σ_{test}^{t+1})

{ 1. Computing the parameters gradients }

{ 1.1 Forward Propagation: }

```

for  $i = 1$  to  $L$  do
   $s_i \leftarrow W_i^{(a)} a_{i-1}$ 
   $a_i, \mu_i, \sigma_i \leftarrow BatchNorm(s_i)$ 
  if  $i < L$  then
     $a_i \leftarrow ReLU(a_i)$ 
  else
     $a_i \leftarrow SoftMax(a_i)$ 
  end if
end for

```

{ 1.2 Backward Propagation: } **for** $i = L$ **to** 1 **do**

```

  if  $i = L$  then
     $g_{a_i} \leftarrow a_i - a^*$ 
  else
     $g_{a_i} \leftarrow ReLUBackward(g_{a_i}, a_i)$ 
  end if
   $g_{s_i} \leftarrow BatchNormBackward(g_{a_i}, s_i, \mu_i, \sigma_i)$ 
   $g_{a_{i-1}} \leftarrow W_i^{(a)\top} g_{s_i}$ 
   $g_{W_i} \leftarrow g_{s_i} a_{i-1}^\top$ 
end for

```

{ 2. Accumulating the parameters gradients }

```

for  $i = 1$  to  $L$  do
   $W_i^{(h),t+1} \leftarrow Update(W_i^{(h)}, \eta, p, g_{W_i})$ 
   $\mu_{test}^{t+1} \leftarrow \varepsilon \mu_{test} + (1 - \varepsilon) \mu_{test}$ 
   $\sigma_{test}^{t+1} \leftarrow \varepsilon \sigma_{test} + (1 - \varepsilon) \sigma_{test}$ 
  if  $count = k$  then
    if VariabilityFlag = True then
       $W_i^{(a),t+1} \leftarrow Transfer(W_i^{(h),t+1})$ 
    else
       $W_i^{(a),t+1} \leftarrow Quantize(W_i^{(h),t+1})$ 
    end if
  end if

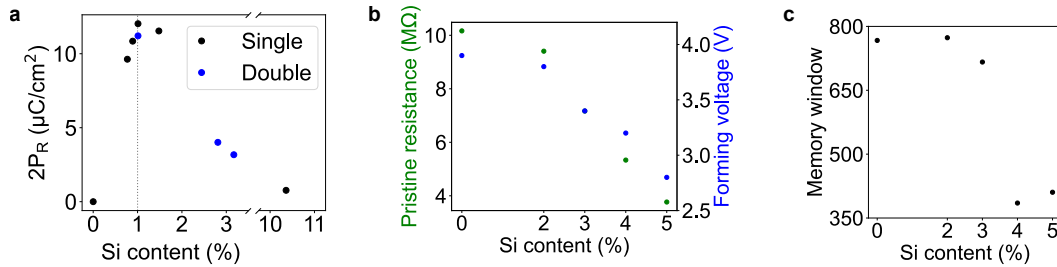
```

```

    end if
  end if
  countt+1 = count + 1
end for

```

Supplementary Note 8: Rationale for Selecting 1% Silicon Doping



Supplementary Figure 6. Si doping optimization (a) Median polarization memory window for all process conditions after 10,000 cycles, extracted using PUND (± 4 V/10 kHz) measurements on a single $100 \times 100 \mu\text{m}^2$ FeCAP device. *Single* and *double* refer to the number of implantation steps. (b) Median pristine resistance (green) and forming voltage (blue) as a function of silicon content. Measurements were performed on a 4 kbit array with programming pulses of 100 ns. (c) Median memory window, defined as the ratio between the median High Resistance State and the Low Resistance State, as a function of silicon content.

This Note provides further details on how we determined that 1% Si doping in 10 nm HfO_2 is the best trade-off for both ferroelectric (FeCAP) and memristive (RRAM) operation.

Ferroelectric enhancement. Suppl. Fig. 6a plots the ferroelectric switching window—as indicated by the median double remanent polarization, $2P_r$, for HfO_2 films doped with various amounts of silicon. We measured $2P_r$ via the Positive-Up-Negative-Down (PUND) method at ± 4 V/10 kHz on a $100 \times 100 \mu\text{m}^2$ FeCAP cell. Up to about 1% Si doping, $2P_r$ steadily increases, indicating stronger ferroelectricity. Beyond this range, raising the Si content no longer enhances—and may even degrade—remnant polarization.

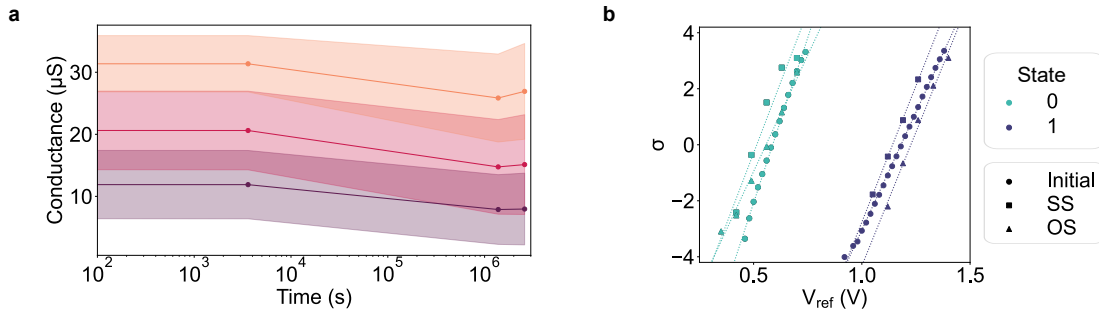
Memristive trade-off. We next investigated how Si doping affects the memristor mode of the same HfO_2 stack, including the forming voltage, pristine resistance, and High/Low Resistance State (HRS/LRS) separation.

As shown in Suppl. Fig. 6b, the pristine resistance decreases from around $10 \text{ M}\Omega$ at 0% doping to $3.7 \text{ M}\Omega$ at 5% doping, making forming easier (forming voltage falls from 3.9 V to 2.8 V). This suggests higher oxygen-vacancy concentration with increased doping, which is favorable for filament formation. However, Suppl. Fig. 6c demonstrates that the ratio HRS/LRS narrows at higher Si content, reducing the dynamic range available for multi-level analog programming. This effect becomes pronounced above 1–2% Si.

Based on these data, it appears that 1% Si robustly boosts ferroelectric polarization without excessive degradation of RRAM voltage/current requirements. Going above 1–2% Si lowers the memristor's HRS/LRS ratio too much, impairing multi-level analog states. We thus selected 1% Si in 10 nm HfO_2 as the optimal solution. Note that the main paper uses a thinner Ti layer (4 nm vs. 10 nm) than the measurements reported in Suppl. Figs. 6b and 6c, which slightly increases both pristine resistance and forming voltage, but does not alter the overall doping trends.

Supplementary Note 9: Retention of analog memristor states and digital FeCAP states of the unified memory

Suppl. Fig. 7a shows that the lowest three conductance levels in our memristor devices remain stable over a period of one month at room temperature: The medium-term retention of analog weights does not pose a significant concern. (We focused on those levels as lower conductance levels are known to have lower retention than higher conductance levels if HfO_x memristors.) Suppl. Fig. 7b shows the zero and one states in a 16-kilo-device FeCAP array ($3\text{ V} - 2\text{ }\mu\text{s}$ reading and programming pulses) before (initial) and after (SS, same state) data retention at 125°C for 1 hour—equivalent to approximately 3 years of retention at room temperature. Moreover, we verified that it is possible to rewrite the cells to the opposite state (OS) after data retention, which has been identified as the limiting factor for data retention due to imprint phenomena³. These findings align with our previously published data on $\text{HfO}_2\text{:Si}$ ferroelectric capacitors⁴.



Supplementary Figure 7. Data retention. **a** One-month evolution of the three lowest conductance levels in our memristor devices at room temperature. **b** Cumulative distributions of the zero and one states in a 16-kilo-device FeCAP array ($3\text{ V} - 2\text{ }\mu\text{s}$ reading and programming pulses) before (initial), after data retention at 125°C for 1 hour (SS, same state), and after rewriting the cells to the opposite state (OS) following data retention. This voltage is measured by ramping up the reference voltage (V_{ref}) of the on-chip sense amplifier.

Supplementary Note 10: Managing Overlapping Conductance Levels and Iterative Programming

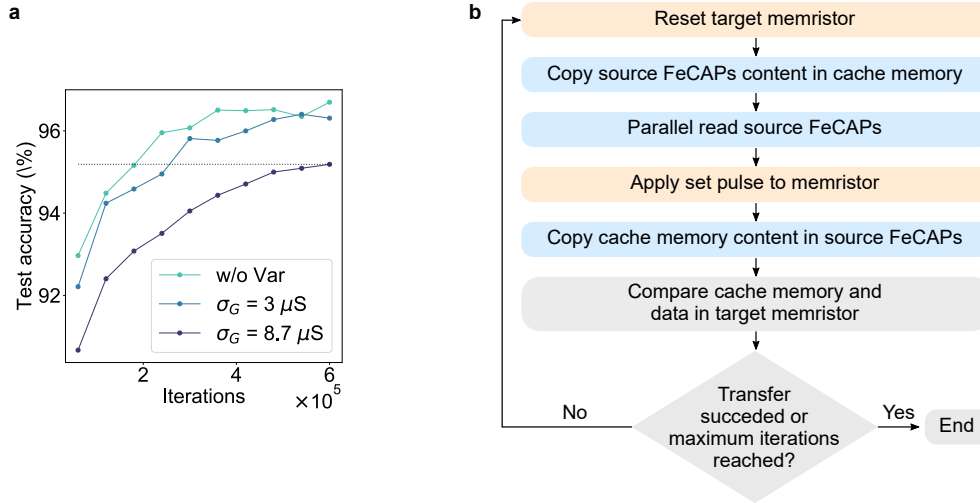
Suppl. Fig. 8a illustrates how the conductance variability of our memristor devices influences accuracy during online training on the MNIST dataset in the online case (i.e., without mini-batches). In these simulations, each analog weight is programmed in a single-shot operation by reading the 3 MSBs from the FeCAPs and applying a single set pulse to the memristor.

This standard deviation matches our measured device variability (see main text, Fig. 4). Under these conditions, the final accuracy after 10 epochs stabilizes around 95.2%. We also artificially lowered the standard deviation to $3\text{ }\mu\text{S}$ to emulate a more uniform device population. This reduces the frequency and severity of “transfer errors,” boosting the final accuracy by ~ 1 percentage point (to $\sim 96.2\%$) and accelerating convergence during training. In that case, the training dynamics is close to the baseline with no variability.

We also note in Suppl. Fig. 8a that achieving the same accuracy level at $\sigma_G = 8.7\text{ }\mu\text{S}$ requires roughly $2.3\times$ more total programming pulses compared with $\sigma_G = 3\text{ }\mu\text{S}$. These additional pulses mostly come from repeated training updates in order to compensate for misprogrammed conductance states.

To mitigate transfer errors, we propose an optional multi-step “iterative” approach to programming the memristor (see conceptual flow in Suppl. Fig. 8b):

1. **Reset.** The target memristor is first reset to a known high-resistance baseline.



Supplementary Figure 8. Impact of the MLC programming accuracy on the test accuracy. **a** Simulated average test accuracy over 5 runs as a function of the number of presented inputs for error-free analog weight updates, as well as for transfer-induced errors with conductance standard deviations $\sigma_G = 8.7 \mu S$ and $\sigma_G = 3 \mu S$. The trained task is MNIST online (see main text). **b** Iterative program-verify scheme exploiting the hybrid memory circuit.

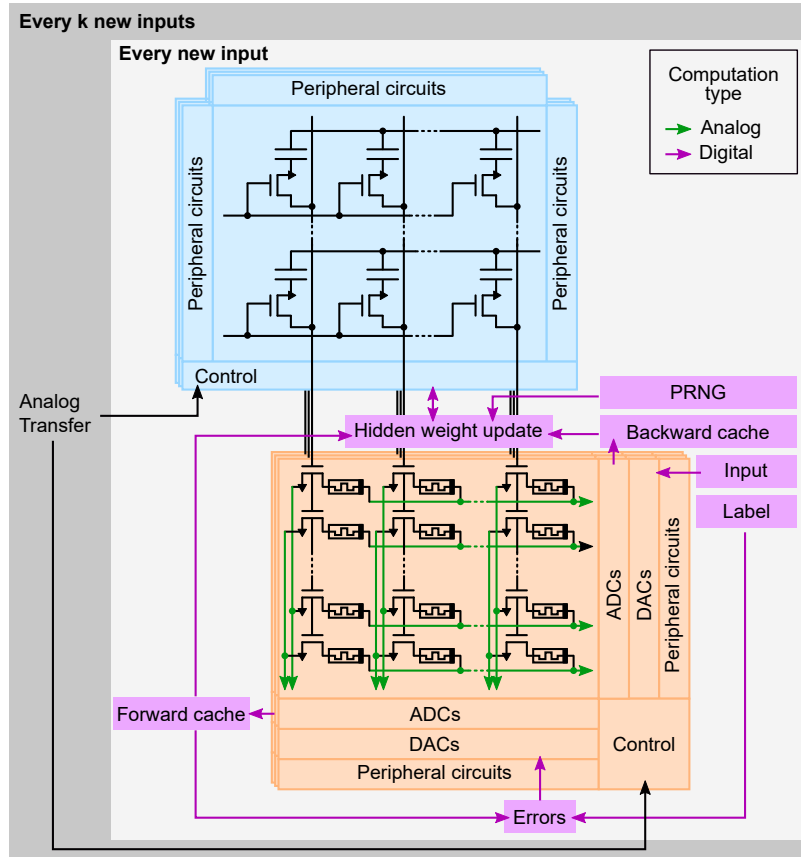
2. **Cache the FeCAP data.** Before programming, the 3 MSBs stored in the FeCAPs are copied to a small volatile cache (e.g., a latch or SRAM cell). This step preserves the original weight value, which may need to be restored if multiple iterations prove necessary.
3. **Set pulse and verification.** A single set operation is performed by applying the FeCAP-derived gate voltage on the selector transistor, while a suitable voltage is applied to the memristor terminals. The device's conductance is then compared against the cached data to check if it falls within an acceptable range.
4. **Iteration or termination.** If the programmed conductance is significantly off-target, the process iterates (i.e., another reset and set cycle), up to a predefined maximum iteration count. Otherwise, if the programmed conductance is acceptable, or the iteration limit is reached, the procedure ends.

Still, the advantage of an iterative programming scheme for training over a single-shot programming strategy is debatable. In the case presented in Suppl. Fig. 8a, if we need more than 2.3 reprogramming attempts on average to effectively shrink σ_G from $8.7 \mu S$ to $3 \mu S$, the scheme may become less efficient overall than single-shot writes (due to increased energy consumption and faster wear-out). Conversely, if fewer than 2.3 attempts suffice, the iterative approach can be beneficial, bringing accuracy closer to the best-case scenario without substantially increasing training overhead.

Supplementary Note 11: Data-flow of training operations

Suppl. Fig. 9 illustrates the data flow of operations during training. Both forward and backward passes are handled analogically through the memristor crossbar, exactly as in standard in-memory computing approaches with memristors⁵⁻⁸.

The only difference here is that, after computing the gradients, weight updates are applied to the “hidden” weight values stored digitally in FeCAPs. Specifically, the backpropagated errors and neuron activations are processed by local digital circuitry, which adjusts the FeCAP-based hidden weights according to our stochastic gradient update scheme at every new input. A small pseudo-random number generator provides the probability mask for updates. Finally, to keep the analog memristors in sync,



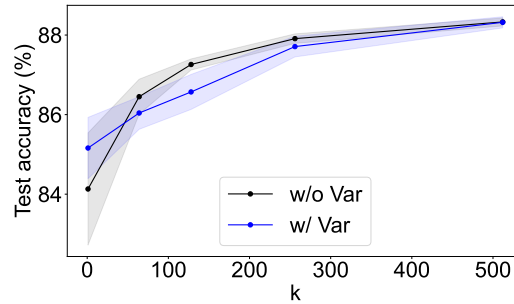
Supplementary Figure 9. Data-flow of training operations. Feed-forward and backward matrix-vector multiplication operations are evaluated with memristors arrays. Hidden weight updates are computed with digital circuits and applied to the FeCAP arrays

we periodically (every k new inputs) transfer the three MSBs from FeCAPs (plus sign) into the memristors, using our direct digital-to-analog transfer method (Fig. 3 in the main text).

Note that this dual-memory design also enables pipelining – while forward/backward passes occur on the memristors, updates can proceed in parallel on the FeCAP array, reducing idle time.

Supplementary Note 12: Impact of variability of the transfer operation in the CIFAR-100/CIFAR-10 training scenario

This Note evaluates the impact of transfer variability on the accuracy for the CIFAR-10 transfer learning task. We ran experiments varying the update period k (i.e., how many samples are processed before transferring weights). Suppl. Fig 10 indicates that at our chosen $k=512$, device variability has negligible impact. For smaller k , frequent transfers amplify device-level noise.



Supplementary Figure 10. Impact of variability of the transfer operation on CIFAR-10 transfer learning task. Test accuracy of the CIFAR-100/CIFAR-10 transfer learning task as function of the update period of the analog weights (k) with (w/ Var) and without (w/o Var) transfer-induced errors. The optimal accuracy is obtained for $k=512$, where the impact of variability is also minimized.

References

1. Okuno, J. *et al.* A highly reliable 1.8 V 1 Mb Hf_{0.5}Zr_{0.5}O₂-based 1T1C FeRAM Array with 3-D Capacitors in 2023 International Electron Devices Meeting (IEDM) (IEEE, 2023).
2. Ramaswamy, N. *et al.* NVDRAM: A 32Gb Dual Layer 3D Stacked Non-volatile Ferroelectric Memory with Near-DRAM Performance for Demanding AI Workloads in 2023 International Electron Devices Meeting (IEDM) (IEEE, 2023).
3. Laguerre, J. *et al.* Data Retention Insights from Joint Analysis on BEOL-Integrated HZO-Based Scaled FeCAPs and 16kbit 1T-1C FeRAM Arrays in 2024 IEEE International Reliability Physics Symposium (IRPS) (2024), 1–7.
4. Francois, T. *et al.* 16kbit HfO₂:Si-based 1T-1C FeRAM Arrays Demonstrating High Performance Operation and Solder Reflow Compatibility in 2021 IEEE International Electron Devices Meeting (IEDM) (IEEE, 2021).
5. Nandakumar, S. R. *et al.* Mixed-Precision Deep Learning Based on Computational Memory. *Frontiers in Neuroscience* **14** (2020).
6. Cai, F. *et al.* A fully integrated reprogrammable memristor–CMOS system for efficient multiply–accumulate operations. *Nature Electronics* **2**, 290–299 (2019).
7. Gokmen, T. & Vlasov, Y. Acceleration of Deep Neural Network Training with Resistive Cross-Point Devices: Design Considerations. *Frontiers in Neuroscience* **10** (2016).
8. Wan, W. *et al.* A compute-in-memory chip based on resistive random-access memory. *Nature* **608**, 504–512 (2022).